

FINAL CHALLENGE

MCR2

Angel Dominguez Franco	A00835367
Nabor Sebastian Toro García	A01028428
Lucas Tavares Quintela Blanc	A01831998
Pablo Díaz Aguilar	A00836765
José Omar Cavazos González	A00840617



Equipo 3

Introduction

Problem Statement: In precision robotics (like package delivery or robotic arms), open-loop motor control is insufficient due to battery fluctuations, friction, and varying loads.

Objective: To develop a robust, distributed control system that maintains a constant motor speed regardless of external disturbances using the ROS 2 ecosystem.

Key Challenges: Translating high-level ROS 2 commands to low-level PWM signals & maintaining real-time constraints.

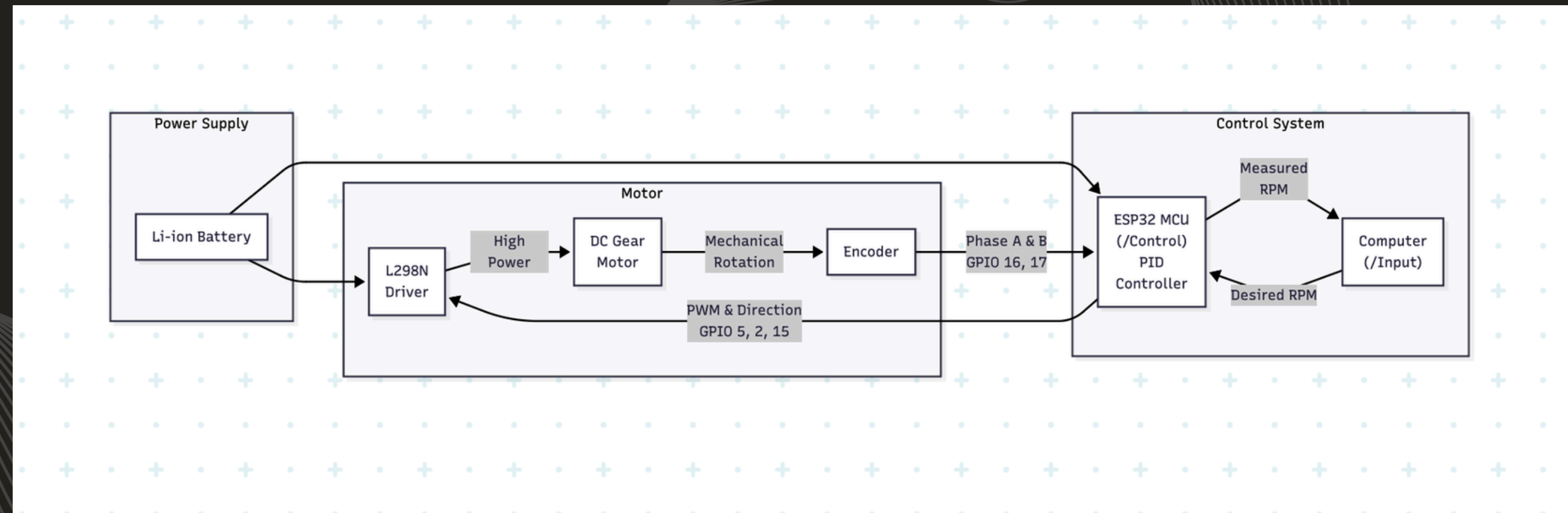


Metodology Hardware

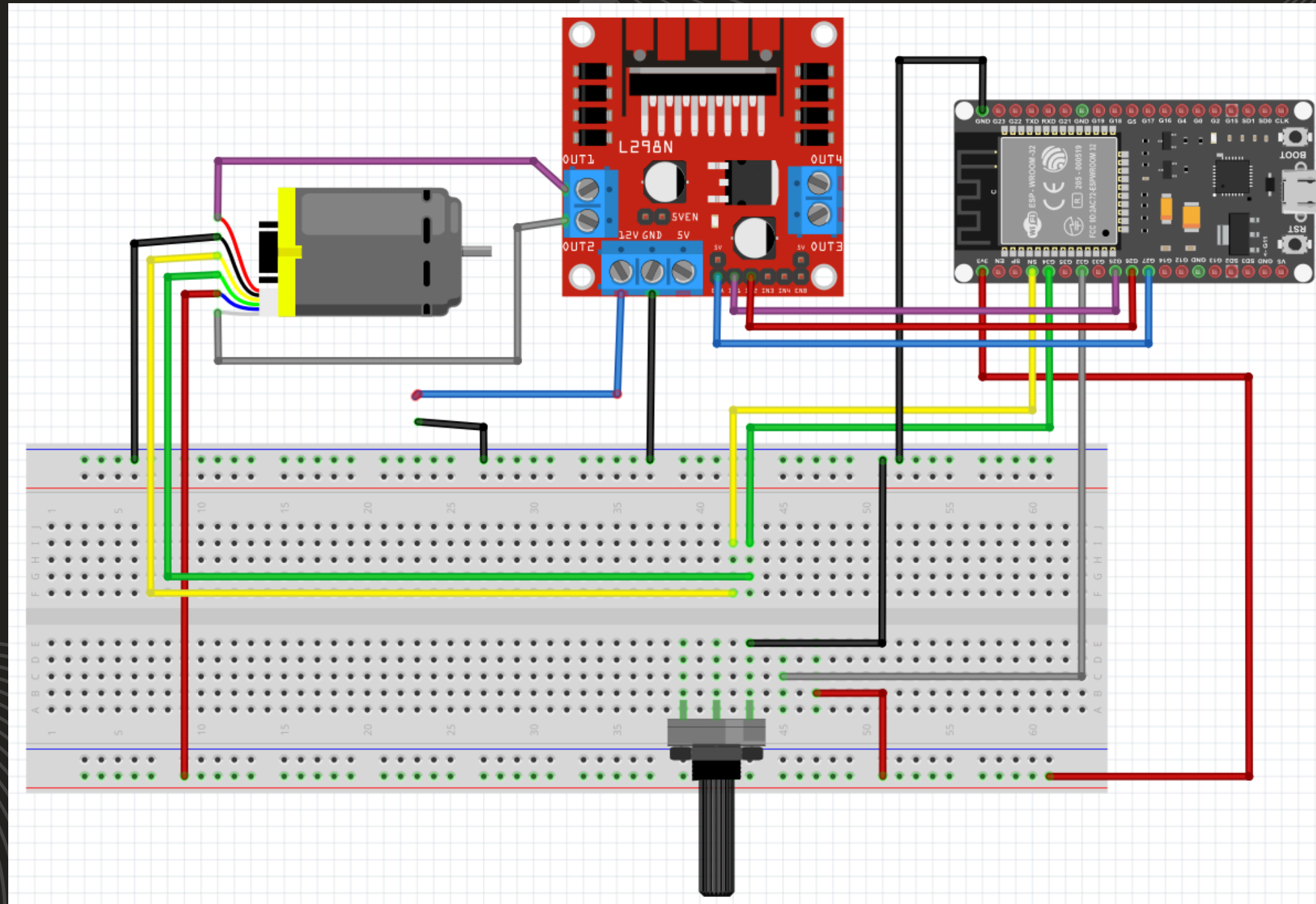
Materials:

- Motor DC
- ESP32
- L298N H-Bridge
- Potenciometer
- Encoder
- Power supply

Block diagram



Implementation (Hardware connection)



Metodology Software

Layers

Middleware

- Communication is handled using the XRCE-DDS protocol through a Serial/USB connection.

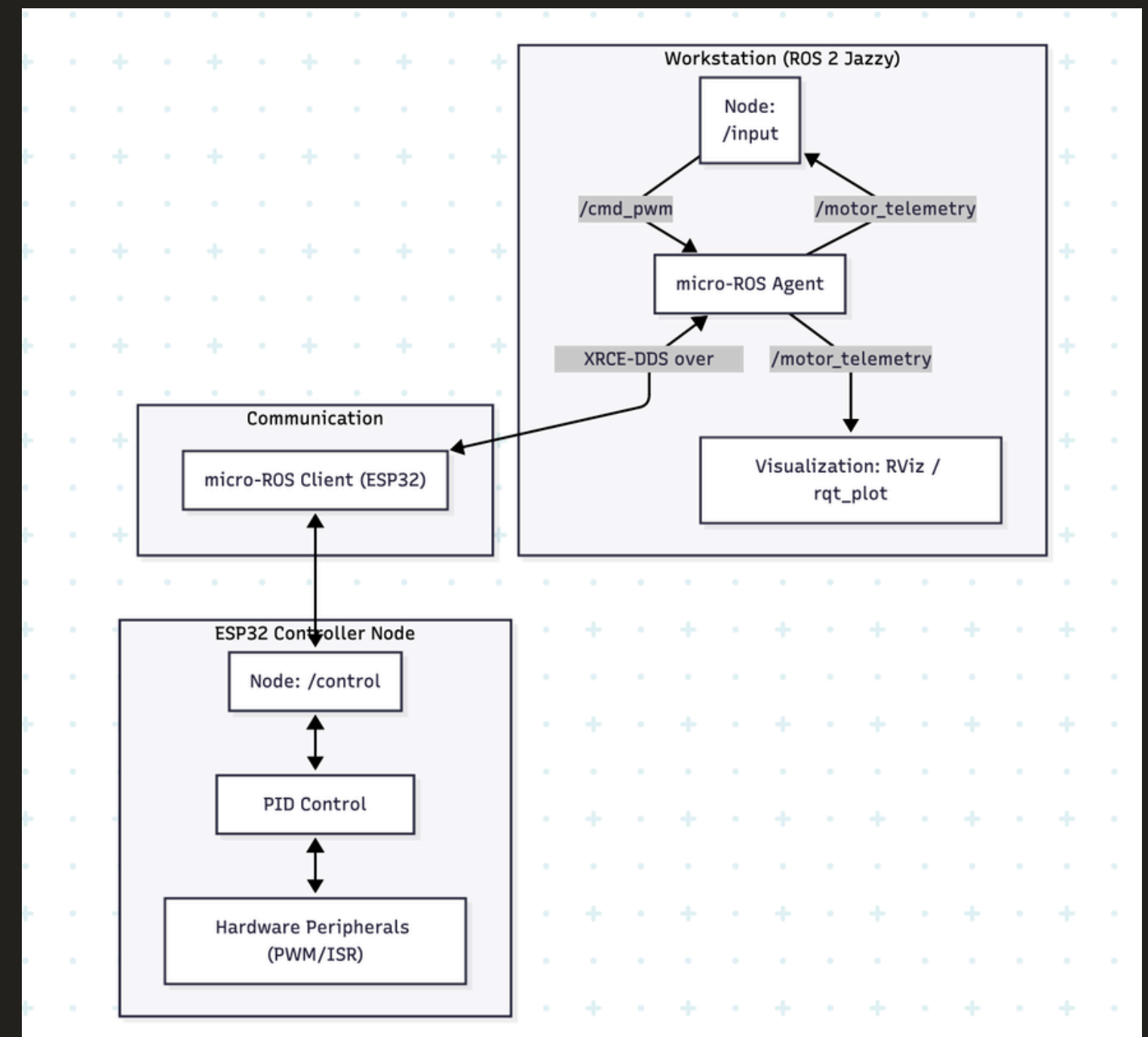
ROS2

- We use the micro-ROS Agent together with the ROS Middleware. The agent acts as a bridge between the microcontroller and the ROS 2 network, translating the XRCE-DDS messages into standard ROS 2 DDS communication.

Observation

- Tools such as rqt_plot and ros2 topic are used to visualize and monitor the system data in real time, such as motor speed, control signals, or sensor values.

Block diagram



Metodology Control

PID controller implementation

Proportional (K_p): 20

Corrects the immediate error.

Integral (K_i): 0.1

Eliminates steady-state error by
accumulating past offsets.

Derivative (K_d): 1.5

Predicts future error to dampen oscillations
and prevent overshoot.

`float Kp = 20.0;`

`float Ki = 0.1;`

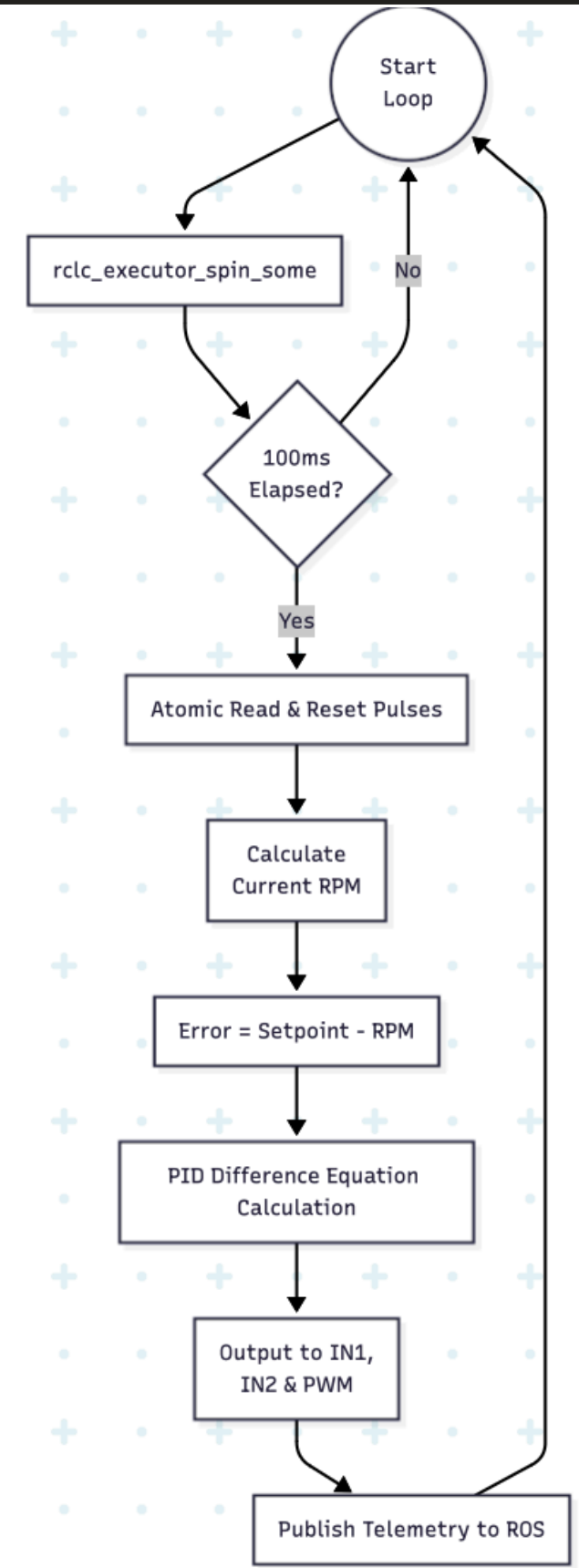
`float Kd = 1.5;`

`int et0 = 0;`

`int et_1 = 0;`

`int et_2 = 0;`

$$u[k] = u[k - 1] + \left(K_p + \frac{K_d}{T_s} \right) e[k] + \left(-K_p + K_i T_s - \frac{2K_d}{T_s} \right) e[k - 1] + \left(\frac{K_d}{T_s} \right) e[k - 2]$$



Implementation (Microcontroller Software)

- The ESP32 reads the quadrature encoder using interrupts to count pulses and detect rotation direction.
- A micro-ROS node is initialized on the ESP32 with one subscriber for `cmd_pwm` and one publisher for `motor_telemetry`.
- Every 100 ms, the encoder pulse count is converted into motor speed in RPM.
- The speed error is processed with a discrete PID controller implemented in difference-equation form.
- The control output is translated into PWM and direction signals and sent to the L298N driver.
- The measured motor speed is then published back to ROS for monitoring and validation.

ROS Communication

- The computer-side ROS node sends the speed reference to the ESP32 through the `cmd_pwm` topic.
 - The ESP32 publishes the measured motor speed back to the computer through the `motor_telemetry` topic.
 - This communication is handled through micro-ROS over a serial connection.
 - ROS tools were used to monitor the response and validate the controller behavior in real time.



Conclusion

- A closed-loop DC motor speed control system was successfully implemented using an ESP32, an encoder, a PID controller, and ROS communication.
- The system was able to receive a speed reference, measure the real motor speed, and adjust the PWM signal to reduce the control error.
- The experimental tests showed that the implementation works correctly and that the motor is able to follow the commanded reference in a stable way.
- Overall, the project validated the integration of hardware, embedded control, and ROS-based communication in a functional solution.

